Programação 3D com Three.js

Pedro Ximenes

Universidade Católica de Pernambuco (UNICAP)

11 de maio de 2026

Sumário

1. De Canvas 2D para Three.js
   - O que você já sabe
   - Por que Three.js?
2. Pilares do Three.js
   - Scene, Camera e Renderer
   - Geometry, Material e Mesh
3. Exemplo 1 – Rotação interativa
   - Esfera e chapa com controles de teclado
4. Exemplo 2 – Grupos e escopo
   - Duas esferas com transformações independentes
5. Exemplo 3 – Sistema solar
   - Hierarquia e animação com tempo
6. Projeto
   - Cubo Mágico 3D

Você já sabe Canvas 2D

Nas aulas anteriores você aprendeu a desenhar e animar com **Canvas 2D**. Essa base é tudo o que você precisa para entender o Three.js:

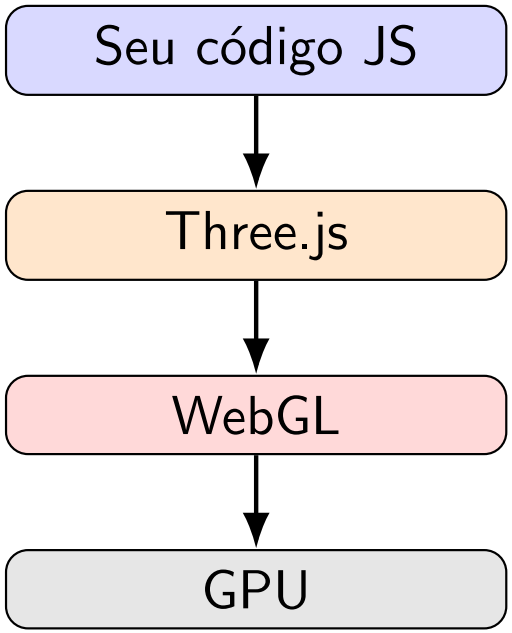
Canvas 2D	**Three.js (3D)**
`ctx.fillRect(...)`	`new THREE.BoxGeometry(...)`
`ctx.arc(...)`	`new THREE.SphereGeometry(...)`
`ctx.translate(tx,ty)`	`mesh.position.set(tx,ty,tz)`
`ctx.rotate(ang)`	`mesh.rotation.y = ang`
`ctx.scale(sx,sy)`	`mesh.scale.set(sx,sy,sz)`
`ctx.save()`	`const g = new THREE.Group()`
`ctx.restore()`	(sair do escopo do grupo)
`requestAnimationFrame`	`requestAnimationFrame`
`addEventListener('keydown',...)`	`addEventListener('keydown',...)`

Conclusão

A **lógica** é a mesma. O que muda é que agora temos **três dimensões** e uma biblioteca que cuida do renderizador por nós.

Por que Three.js?

- O navegador suporta 3D através do **WebGL**
- WebGL puro exige escrever shaders em GLSL — complexo demais para começar
- **Three.js** é uma camada sobre o WebGL que oferece uma API amigável para JavaScript
- Roda em qualquer navegador moderno, sem instalação
- Enorme comunidade, documentação rica e exemplos prontos



Os três pilares

Todo projeto Three.js tem **três componentes obrigatórios**:

Scene

Contêiner de todos os objetos

Como a "tela em branco" do Canvas: você adiciona objetos nela.

Camera

Ponto de vista do observador

Define de onde e como a cena é enxergada (perspectiva ou ortográfica).

Renderer

Gera a imagem no `<canvas>`

Renderiza a cena pela câmera a cada frame, como o `ctx` do Canvas.

Fluxo básico

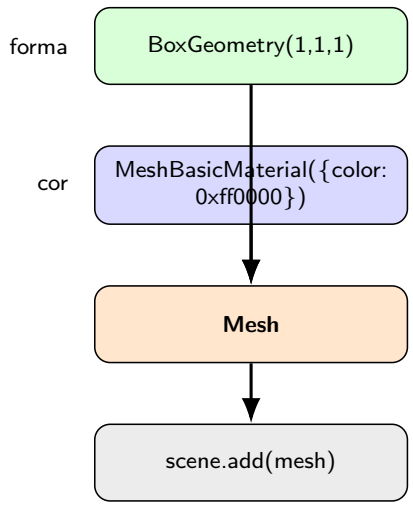
`scene` ← adicionar objetos → `renderer.render(scene, camera)`

Geometry + Material = Mesh

Um objeto 3D no Three.js é sempre um **Mesh**:

$$\text{Mesh} = \text{Geometry} + \text{Material}$$

- **Geometry**: forma geométrica (cubo, esfera, cilindro...)
- **Material**: aparência (cor, wireframe, textura...)
- **Mesh**: o objeto final adicionado à cena



Análogo ao Canvas

É como definir o `fillStyle` (material) e depois chamar `fillRect()` (geometry) – mas em 3D.

Setup básico de uma cena Three.js

```
// 1. Incluir a biblioteca no HTML:
// <script src="https://cdn.jsdelivr.net/npm/three@0.165.0/build/three.min.js
    ">

// 2. Criar os tres pilares
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
                 45, // campo de visao (graus)
                 window.innerWidth / window.innerHeight, // aspecto
                 0.1, 1000); // near / far
camera.position.set(0, 2, 8); // posicionar a camera

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setClearColor(0x111111); // cor de fundo
document.body.appendChild(renderer.domElement); // adicionar o canvas

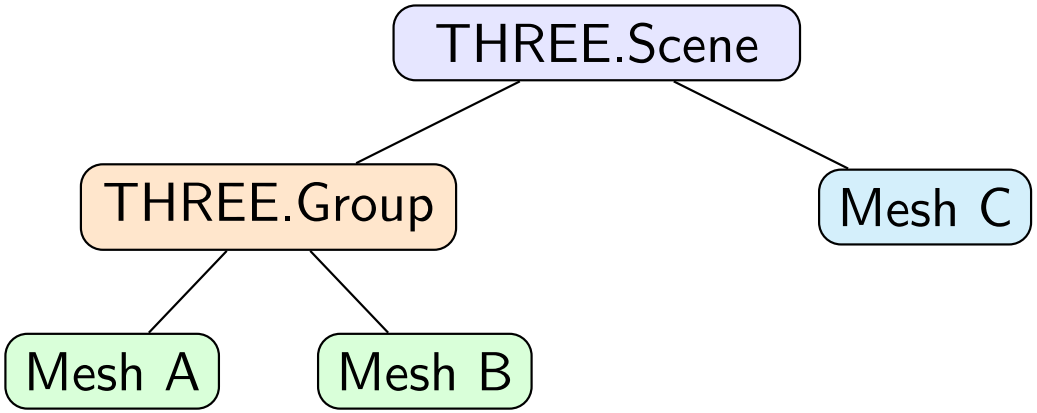
// 3. Loop de animacao (igual ao Canvas!)
function animate() {
  requestAnimationFrame(animate);
  renderer.render(scene, camera);
}
animate();
```

Hierarquia de objetos

Adicione objetos com `scene.add(mesh)` ou agrupe com `THREE.Group` para controlar escopos de transformação – exatamente como `ctx.save()/restore()` no Canvas.

Hierarquia e escopo de transformações

Assim como `ctx.save()/restore()` no Canvas controlam o escopo das transformações, em Three.js usamos **Groups**:



- Transformar o `Group` afeta **todos os filhos** (Mesh A e B)
- Transformar `Mesh C` afeta **só ele**
- Grupos podem ser **aninhados**: filho de um grupo pode ser outro grupo

Exemplo 1 – O que veremos

Cena

Uma **esfera wireframe** amarela acima de uma **chapa azul achatada**.
Os dois objetos estão num mesmo `Group` e se movem juntos.

Controles:

- $\leftarrow$ $\rightarrow$: rotacionar em y
- $\uparrow$ $\downarrow$: rotacionar em x
- **Page Up**: zoom in (escala +10%)
- **Page Down**: zoom out (escala -10%)

Conceito demonstrado:

- Variáveis JS controlam as transformações
- Loop de animação aplica as transformações a cada frame
- `Group` propaga para todos os filhos

Arquivo: `ex1-rotacao.html`

Exemplo 1 – Criando os objetos

```
const grupo = new THREE.Group();
scene.add(grupo);

// Esfera wireframe amarela (raio 1, 20x20 faces)
const esfera = new THREE.Mesh(
  new THREE.SphereGeometry(1, 20, 20),
  new THREE.MeshBasicMaterial({ color: 0xffff00, wireframe: true })
);
grupo.add(esfera);

// Chapa azul: BoxGeometry achatada em Y (0.2), larga em X e Z
const chapa = new THREE.Mesh(
  new THREE.BoxGeometry(4, 0.2, 4),
  new THREE.MeshBasicMaterial({ color: 0x0044ff })
);
chapa.position.y = -1.1; // posicionar logo abaixo da esfera
grupo.add(chapa);
```

BoxGeometry vs escala

Preferimos passar as dimensoes direto na `BoxGeometry` em vez de usar `scale` – isso deixa o codigo mais claro.

Exemplo 1 – Controles de teclado

```
let alfa = 0, beta = 0, delta = 1;

document.addEventListener('keydown', (e) => {
  switch (e.key) {
    case 'ArrowRight': beta -= 2; break; // rotacao em Y
    case 'ArrowLeft': beta += 2; break;
    case 'ArrowUp': alfa -= 2; break; // rotacao em X
    case 'ArrowDown': alfa += 2; break;
    case 'PageUp': delta *= 1.1; break; // zoom in
    case 'PageDown': delta *= 0.9; break; // zoom out
  }
});

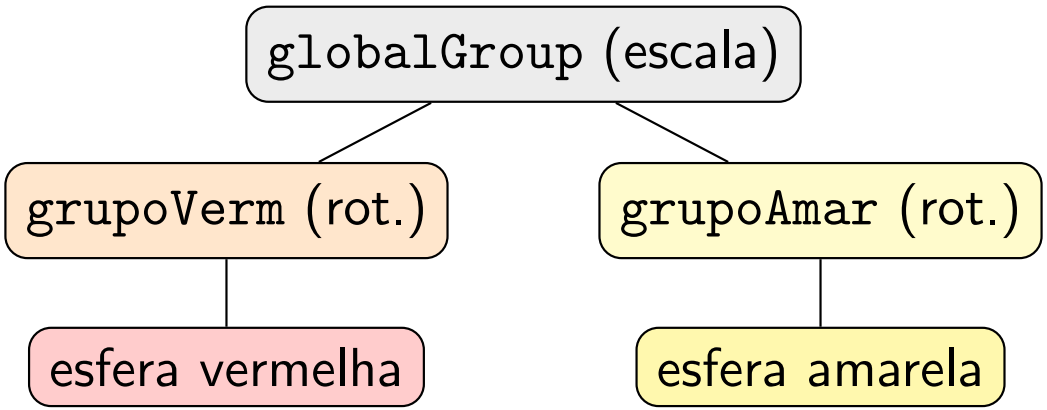
function animate() {
  requestAnimationFrame(animate);
  const toRad = THREE.MathUtils.degToRad;
  grupo.rotation.x = toRad(alfa); // aplica rotacao acumulada
  grupo.rotation.y = toRad(beta);
  grupo.scale.setScalar(delta); // escala uniforme nos 3 eixos
  renderer.render(scene, camera);
}
animate();
```

Exemplo 2 – Grupos e escopo

Problema

Como rotacionar dois objetos de forma **independente** e ainda aplicar uma escala **global** aos dois ao mesmo tempo?

Solução: hierarquia de grupos.



- Cada esfera tem seu próprio grupo: as rotações são **locais**
- A escala no `globalGroup` afeta **ambas** as esferas

Arquivo: `ex2-escopo.html`

Exemplo 2 – Código completo

```
// Grupo global: a escala aqui afeta as duas esferas
const globalGroup = new THREE.Group();
scene.add(globalGroup);

const geo = new THREE.SphereGeometry(1, 20, 20);

// -- Esfera vermelha (posicao x = +3) --
const grupoVerm = new THREE.Group();
const esfVerm = new THREE.Mesh(geo,
  new THREE.MeshBasicMaterial({ color: 0xff3333, wireframe: true }));
esfVerm.position.x = 3;
grupoVerm.add(esfVerm);
globalGroup.add(grupoVerm);

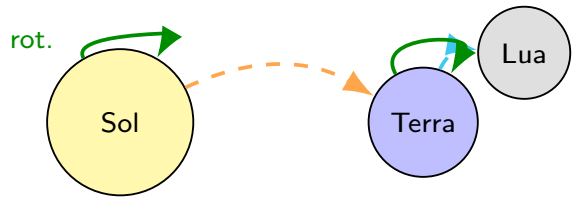
// -- Esfera amarela (posicao x = -3) --
const grupoAmar = new THREE.Group();
const esfAmar = new THREE.Mesh(geo,
  new THREE.MeshBasicMaterial({ color: 0xffff00, wireframe: true }));
esfAmar.position.x = -3;
grupoAmar.add(esfAmar);
globalGroup.add(grupoAmar);

// No loop: rotacao local de cada grupo, escala no global
grupoVerm.rotation.set(toRad(alfa), toRad(beta), 0);
grupoAmar.rotation.set(toRad(alfa), toRad(beta), 0);
globalGroup.scale.setScalar(delta);
```

Exemplo 3 – Sistema solar simplificado

Simplificações usadas:

- Sol gira em torno do próprio eixo a cada **28 dias**
- Terra orbita o Sol em **360 dias**, e gira em **24 h**
- Lua orbita a Terra a cada **28 dias**
- Tamanhos e distâncias não proporcionais
- Objetos em wireframe (iluminação ainda não foi estudada)



Hierarquia

`scene` → `grupoSol` → `grupoTerra` → `grupoLua` → `lua`

Arquivo: `ex3-solar.html`

Exemplo 3 – Hierarquia dos objetos

```
// Sol: na origem, raio 1.2
const grupoSol = new THREE.Group();
scene.add(grupoSol);
const sol = new THREE.Mesh(new THREE.SphereGeometry(1.2,32,32),
  new THREE.MeshBasicMaterial({color:0xffdd00, wireframe:true}));
grupoSol.add(sol);

// Terra: filho do grupoSol -- orbita junto quando grupoSol gira
const grupoTerra = new THREE.Group();
grupoSol.add(grupoTerra);
const terra = new THREE.Mesh(new THREE.SphereGeometry(0.5,20,20),
  new THREE.MeshBasicMaterial({color:0x2244ff, wireframe:true}));
terra.position.x = 5; // distancia ao Sol
grupoTerra.add(terra);

// Lua: filho de grupoLua (que fica no centro da Terra)
const grupoLua = new THREE.Group();
grupoLua.position.x = 5; // mesma posicao da Terra
grupoTerra.add(grupoLua);
const lua = new THREE.Mesh(new THREE.SphereGeometry(0.18,12,12),
  new THREE.MeshBasicMaterial({color:0xaaaaaa, wireframe:true}));
lua.position.x = 1.2; // raio da orbita lunar
grupoLua.add(lua);
```

Exemplo 3 – Loop de animação com tempo

```
let hora = 0, dia = 0;
const toRad = THREE.MathUtils.degToRad;

function animate() {
  requestAnimationFrame(animate);

  hora += 1; // 1 frame = 1 hora
  if (hora >= 24) { hora = 0; dia += 1; }
  if (dia >= 360) dia = 0;

  // Sol: rotacao propria (1 volta a cada 28 dias)
  grupoSol.rotation.y = toRad(dia * 360 / 28);

  // Terra: orbita em torno do Sol (1 volta/360 dias)
  grupoTerra.rotation.y = toRad(dia);

  // Terra: rotacao propria em torno do eixo (24 horas)
  terra.rotation.y = toRad(hora * 15); // 15 graus/hora

  // Lua: orbita a Terra (28 dias)
  grupoLua.rotation.y = toRad(dia * 360 / 28);

  renderer.render(scene, camera);
}
animate();
```

Double buffering

O `WebGLRenderer` usa double buffering **automaticamente**. Não é necessária nenhuma configuração extra.

Projeto – Cubo Mágico 3D

Descrição

Criar uma simulação interativa de um **cubo mágico** (Rubik's Cube) em Three.js, com câmera 3D e animação suave ao girar as faces.

Requisitos mínimos:

- Cubo $3 \times 3 \times 3$ construído com 27 cubinhos coloridos
- Rotação de ao menos **2 faces** (ex.: Face superior e face frontal)
- Animação suave (interpolação do ângulo)
- Câmera: órbita com mouse (usar `OrbitControls`)
- Cores corretas por face

Extras (pontuam mais):

- Todas as 6 faces rotacionáveis
- Botão "embaralhar" com animação
- Detecção de vitória
- Contador de movimentos
- Temas de cor customizáveis

Dica de implementação

Use um `THREE.Group` por face. Ao girar a face, adicione os cubinhos corretos ao grupo, anime a rotação e depois **ancore novamente** os cubinhos de volta à cena.

Projeto – Entregáveis e Avaliação

Informações gerais

- **Grupos de X integrantes**
- **Prazo de entrega**: 26 de maio de 2026 (até o início da aula)
- **Apresentação**: 26 de maio de 2026 (na aula)

O que entregar:

1. **GitHub** do projeto (obrigatório)
   - `README.md` explicando o projeto
   - Commits de **todos** os X integrantes
   - `index.html` executável no navegador
2. **Relatório** em PDF
   - O que foi implementado
   - Dificuldades encontradas
   - Divisão de tarefas

Composição da nota:

Funcionamento	50%
Qualidade do código	25%
Apresentação	**25%**

Regras da apresentação

- Ausência no dia ⇒ **0% na apresentação**
- 1 integrante será sorteado para responder perguntas
- Se não souber ⇒ **0%** e sorteio de outro